UNIVERSITY OF DAR ES SALAAM

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND TELECOMMUNICATIONS ENGINEERING

FINAL YEAR REPORT END SEMESTER TWO

PROJECT TITLE: DESIGN AND IMPLEMENTATION OF AN AUTOMATIC ID CARD ISSUANCE SYSTEM

Student's Name:	**Samuel Dan Musyani**
Registration Number:	**2022-04-09050**
Supervisor's Name:	**Dr. Hashimu Iddi**

ABSTRACT

Manual student ID card distribution at University scale suffers from operational inefficiencies such as extended queue times, narrow service windows confined only to working hours, and a significant administrative burden on staff. This project addresses these constraints through an automated self service kiosk system that retrieves and dispenses pre-printed student identification cards on demand.

The implementation comprises a distributed two tier architecture: a Raspberry Pi 5 serving as a high – level controller for user interface, computer vision processing, authentication logic and database management; and an Arduino Mega 2560 operating as a real time slave controller for mechanical actuation. Communication between controllers occurs via SPI protocol using a two-byte ASCII frame format. The software pipeline includes an optical character recognition (OCR) module for card registration number extraction, two – factor authentication using SMS delivered one time passwords (OTP) with BRIQ Solutions for SMS gateway delivery. Mechanical actuation employs a carousel based circular storage chamber with servo motor control for card positioning and ejection.

The system demonstrates functional OCR detection with validation against institutional registration number formats, complete authentication workflows with bcrypt secured credential storage and configurable lockout policies, and a fully implemented SQLite database schema supporting audit complicance. The software and hardware integration delivers a scalable, auditable ID card distribution system suitable for deployment across University environments.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

ABBREVIATIONS	LONG FORM
2FA	2-Factor Authentication
API	Application Program Interface
CAD	Computer Aided Design
CLK	Clock
CS	Chip Select
DB	Database
GB	Gigabyte
ID	Identification
IoT	Internet of Things
MISO	Master In Slave Out
MOSI	Master Out Slave In
OCR	Optical Character Recognition
OS	Operating System
OTP	One Time Password
PVC	Poly-Vinyl Chloride
PWM	Pulse-Width Modulation
RAM	Random Access Memory
SMS	Short Message Service
SPI	Serial Peripheral Interface
Wi-Fi	Wireless Fidelity

CHAPTER ONE: INTRODUCTION

1.1 BACKGROUND

Universities commonly issue student ID cards that serve as proof of registration and campus identification for various services to students. At the University of Dar es Salaam, the card collection process, which occurs immediately after students have completed registration, involves queuing at the responsible office for collection. However, for a high volume of students, this results in long waiting times, narrow service windows (only operating during working hours), and staff exhaustion.

Therefore, the project aims to develop a prototype of an **Automated ID Card Issuance System** that sorts and gives out pre-printed ID cards to students on demand at all times. The system integrates a microcontroller that controls actuators in real-time, a single-board computer that performs computer vision (for ID identification), student's identity authentication, logging of various activities, and back-end integration. The intended solution aims to reduce distribution time while ensuring secure ID card distribution as it is.

1.2 PROBLEM STATEMENT

The current student ID card issuance process at University of Dar es Salaam involves a waiting period of approximately one week after registration due to delayed card availability in the individual's college. After release, the card collection also requires students to queue and sort a bunch of ID cards. So, for a high volume of students, the staff responsible suffers significant work strain. On top of that, fixed hours also hinder the sorting capacity of students' issuance per time. As a result, card issuance for registered students takes up to several days, which may bring inconvenience to students.

1.3 MAIN OBJECTIVE

Design and implementation of an Automated ID Card Issuance System for pre-printed student ID cards with secure authentication and audit logging while serving students at all times.

1.4 SPECIFIC OBJECTIVE

1. To implement a reliable control system for actuators using a microcontroller capable of selecting and dispensing a single card from a pile of cards in a stored compartment.
2. To implement a computer board application that gives out authentication OTPs, verifies the student's set pin, receives the user registration data, verifies it against a database, and communicates the command to the microcontroller.

3. To implement robust logging of each transaction details in database for auditing and debugging.
4. To implement security policy that uses SMS-delivered OTPs as authentication mechanism, and also enforce measures to deal with invalid attempts and communication to system admin.
5. To evaluate system performance using metrics such as average card retrieval time, authentication success, error handling and dispense rate.
6. To produce a formal report involving design of the prototype at various milestones.

1.5 SIGNIFICANCE OF STUDY

- Operational efficiency: the proposed system reduces long queues and waiting times by allowing students to retrieve ID cards on demand without any staff intervention.
- Reduced administrative burden: the system performs automation which replaces the repetitive tasks done by a university staff allowing them to focus on higher level tasks.
- Scalability, accessibility and replicability: Although the proposed system is implemented as a prototype for UDSM, the system can be adopted by other institutions such as banks for card applications. Also, access to students retrieving their ID cards beyond standard normal working hours.
- Academic contribution: the project demonstrates practical application of embedded systems, back-end software and other mechanical design theory and automation studied as theory in various academic year(s).

1.6 SCOPE AND LIMITATIONS

This project focuses on the design and implementation of an **Automated ID Card Issuance System** which integrates the software part (back-end, user interaction and computer vision verification) and hardware part with microcontroller and actuators for card sorting and dispensing the said card. Therefore, the prototype does not implement on-device printing of the PVC cards, mobile application or payment gateway.

The system intends to use a mock database for testing unless institutional access is granted.

CHAPTER TWO: LITERATURE REVIEW

In this chapter, various existing systems, projects and technologies related to the proposed project are reviewed. The purpose of the review is to understand different approaches, scope, limitations and gaps and hence, establish basis of the proposed project. Therefore, the following points gathered below feature ideas with similar concepts as the proposed project.

2.1 RELATED PROJECTS

- Automatic and Smart Vending systems: these are some self-service systems that provide mostly goods (foods) to users without much human intervention, mostly observed in supermarkets for snacks and beverages. These systems have two categories: non-IoT machines, which operate by giving physical cash to the machine, and IoT machines, which work with cashless payments and track the availability of stock, locations to satisfy customer preferences by using microcontrollers (Ratnasri & Sharmilan, 2021). Also, some vending systems apply image processing with deep learning to detect and categorize various target objects of interest in the image. These kinds of vending machines also implement a weight data acquisition part to increase the accuracy of target object detection (Xia, K., Fan, H., Huang, J., Wang, H., Ren, J., Jian, Q., & Wei, D).
- System architectures and control strategies: in most of the dispensing applications, generally a microcontroller is used with a communication module, mostly Wi-Fi, with a logic to directly manage actuator control, sensor inputs, safety interlocks, and communicate with the server or a computer for other complex tasks (Prasad, Kumar, Shriram, Prof. Dr. Mane, 2024).
- Authentication and security: this is one of the most critical part of the system that deals with sensitive items. In automated environments, combining multiple verification layers (e.g OTP, biometrics, card scanning) improves security. In ATM systems, work on authentication explores multi-factor approaches, integrating OTPs with biometric factors to reduce unauthorized access risk (Kibaya, Lubobya, 2024).
- Automated pharmaceutical dispensing systems: feature customer interaction stations with graphical interfaces, drug verification stations, pill counting, packaging stations, and controllers managing communication with external systems and databases.
- Intelligent vending machines with computer vision:AI vending machines use real-time computer vision registering item removals with roughly 99% recognition accuracy across product categories. Systems recognize customers using facial recognition, recommend

products based on purchase history, accept multiple payment types, monitor stock remotely, and self-diagnose technical faults with alerts to operators.

- Automated Library book retrieval and return systems: Library management robots use ESP32 microcontrollers processing sensor data in real time, servo motors for robotic arms with grippers, and RFID readers confirming book identity before retrieval; users request books through mobile applications. Unattended book management systems employ PLC-controlled mechanical hands, gas cylinders for motion control, and RFID detection at retrieval windows; failed detection triggers reversal of book basket to temporary storage (J Kumar, Nanditha V, Pranav P, Sona S, Mary Alex, 2025).

2.2 RESEARCH GAP

From the related projects, the proposed system aims to cover the gap that despite an extensive research on vending and kiosk systems, limited number of projects addresses secure low-cost automated distribution of pre-printed identification cards particularly in University environments.

CHAPTER THREE: METHODOLOGY

3.1 OVERVIEW

The Automatic ID Card Issuance System was developed using a prototype-based, modular approach. Nonetheless, the work was completed in a well defined order that began with issue identification, function definition, system division into smaller modules, development and testing of each module, and integration of the modules into a single prototype. This strategy made sense since the project integrates electronic control, mechanical movement, and software.

The user interface, computer vision, authentication, database, controller communication and motor control were treated as separate modules and then interconnected.

3.2 PROBLEM IDENTIFICATION AND REQUIREMENT

The first step was to analyze the current process of student ID card collection and identify its main weaknesses. Usually students have to wait until the cards get to their colleges and then queue at an office during working hours. This results in long wait times, congestion and repeated work for staff. This problem defined the system's requirements. The proposed kiosk needed to recognise pre-printed cards, link each card to the correct student, store the location of the card, inform the student, authenticate the person collecting it, release the correct card and record all important activity.

The scope of the project was limited to issuance of cards that were already printed. The card printing, the payment services and a mobile application were excluded. A mock university database was used during development because direct access to the official institutional database was not guaranteed.

3.3 SYSTEM’S MODULES

After the definition of the requirements, the system was divided in a high level control section and a low level control section. The top layer runs on a Raspberry Pi and takes care of the user interface, student verification, card-image processing, authentication, database access, notifications and logging of transactions. The low-level section is based on an Arduino Mega and is designed to control the motors and other physical components used to position and release cards.

This separation allows the Raspberry Pi to decide and the Arduino to perform time sensitive physical actions. The whole design was divided into card registration, card storage, student notification, authentication, card retrieval, database management, communication and audit logging. The modular design gave the opportunity to develop and correct one part without re-building the whole system.

3.4 CARD IDENTIFICATION, REGISTRATION AND STORAGE

On the staff side, the implementation starts when the pre-printed student cards are placed in the kiosk. A camera takes a photo of each card. The image is processed to find the card, adjust the rotation, and zoom on the part containing the registration number. Then optical character recognition is used to read the registration number as it is unique to each student. The number extracted is checked for the expected university format and checked on the mock university database.

If the record is valid, the system retrieves the basic information of the student and writes the card to an available location in the storage chamber. The relationship between the student and the card and its storage position is saved in the local database. Cards that cannot be read or verified are rejected or marked for staff attention. This process ensures that the kiosk knows which card belongs to each student and where it is stored.

3.5 STUDENT NOTIFICATION AND AUTHENTICATION

After successful registration of card, the system will generate the one-time password and send it to the student through SMS and email. A first-time user may also be given a temporary PIN. During collection, starting the process from the touchscreen the student enters the registration number.

The system checks the student is valid, a card is available, the card has not been previously collected and the account is not locked. The student then enters the one time password. The first-time user also confirms the temporary PIN and sets up a personal permanent PIN. The combination of registration number with a one-time password and PIN provides more than a single level of identity verification. Wrong attempts are limited in number and multiple failures lead to temporary lockout.

3.6 USER INTERFACE AND DATABASE IMPLEMENTATION

To help the student navigate the collection process, an interface was created. Welcome, registration number entry, one-time password entry, PIN creation or verification, waiting for card dispensing, and final confirmation are all displayed in the same order as the actual process. When a student record is unavailable, a card has already been collected, authentication information is inaccurate, or the account is locked, the user is notified via separate error screens.

The interface uses large controls that are appropriate for a small kiosk display and offers one task at a time. Student records, card locations, authentication details, collection status, batches, and audit events are all stored in a local database that supports the interface. The audit log and stored status are updated upon card collection.

3.7 HARDWARE CONTROL AND MECHANICAL PROTOTYPE

A circular card-storage chamber with indexed places is used in the physical design. Every position has a corresponding slot in the database. The Raspberry Pi locates the designated slot and issues an instruction to the Arduino Mega after a student has been verified. The motors that move the chamber and release the chosen card are controlled by the Arduino.

Basic movement and card-ejection motions were tested using servo motors. Before the entire dispensing mechanism was completed, the communication and motor-control concepts were tested because the hardware part was created as a prototype. The dispensing screen in the current software interface depicts the anticipated background mechanical movement.

3.8 INTEGRATION AND TESTING

The various modules were gradually integrated once they were operational. In order to verify the extracted registration numbers, the computer vision module was linked to the student database. After that, card records were connected to storage locations, and authentication credentials were sent via the notification module. While communication methods were ready to transmit card-selection commands to the Arduino, the user interface was linked to the database and authentication modules.

Card detection and registration-number extraction were tested using sample card images. Authentication, attempt limitations, and lockout behavior were tested using both valid and invalid one-time passwords and PINs. Database testing verified logging, card assignment, record insertion, and collection updates. The low-level controller's ability to receive commands and control actuators was validated by communication and servo tests.

CHAPTER FOUR: SYSTEM DESIGN

This chapter reviews various procedures to design, build and evaluate the performance of the prototype. It covers system architecture, hardware and software selection together and security.

4.1 SYSTEM ARCHITECTURE

The system's control architecture as shown in the Figure 4.1 below depicts are planned isolation of two controllers. The high-level controller will be used to control the user interface, network connectivity, 2FA for OTP and student's pin, database access, computer vision verification, transaction logging, and command issuing to the real-time controller. Then, a real-time controller will be used to directly control the motor, safety monitoring (limit switches), status reporting, and mechanical card sorting and ejection.

The pre-printed cards will be organized in indexed storage sections, which will be mapped to existing barcodes on the ID cards. Local (mock) database will be used to replicate the University DB scheme, and testing will emulate that unless permission is granted to access the database (API).

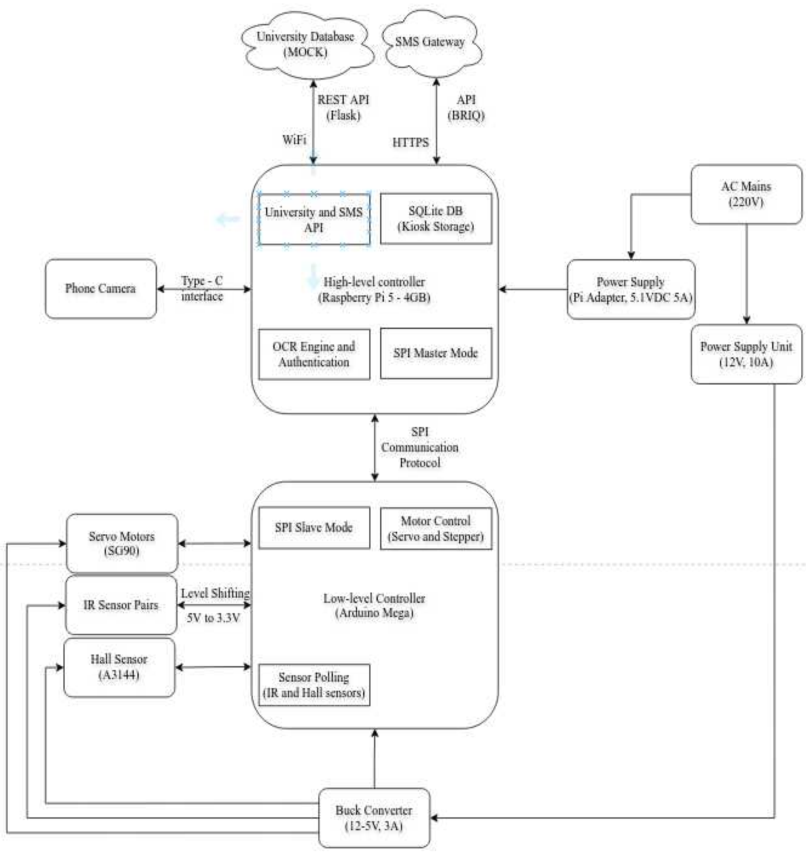


Figure 4.1: System's block diagram

4.2 FUNCTIONAL WORKFLOW

The system's workflow is divided into two distinct parts which are staff side and student side of the workflow. The two sides are clearly described in Figures 4.2 and 4.3 respectively, with the earlier explaining more on the first two phases, preload and notification phase, and the second figure elaborating on retrieval phase.

- Preload phase: responsible staff inserts a bunch of pre-printed cards into the dispenser, then the machine runs a routine scan and maps each card into a slot index in the dispenser DB. The high-level controller records the mapping with the individual card.
- Notification phase: when the cards are available, right after the routine scan, the high-level controller initiates triggering of SMS to particular students. The SMS includes an OTP valid for one retrieval attempt. If it is the particular student's first time use, and extra different and temporary pin is sent via SMS for 2FA.
- Retrieval phase: First, students scan their expired card, and their information is shown on the screen (registration number, name, and degree programme), then an OTP prompt follows, as received from the SMS. Thereafter, the high-level controller validates the OTP and prompts for the student's pin for extra layer of authentication. If it is the student's first time use, after the prompt, the system gives a form to register a new custom pin which will be permanent for each individual.

After that, high-level sends the dispense command to the low-level controller to start the card dispensing process. The low-level microcontroller executes the motion to retrieve the card and verifies the card and its presence, and performs ejection while updating and returning the status to the high-level controller. Lastly, the high-level controller logs the transaction to the database as the audit logs.

- Security measures: after 3 invalid attempts for the same registration number within the same session, the system flags that particular number as locked for amount of time, notifies the system admin and returns to the home screen. In addition to that, in case of any physical tampering, the system alerts the admin and shuts down until manual admin reset is performed.

Below are the functional workflows featuring both student-side and staff-side access of the system:

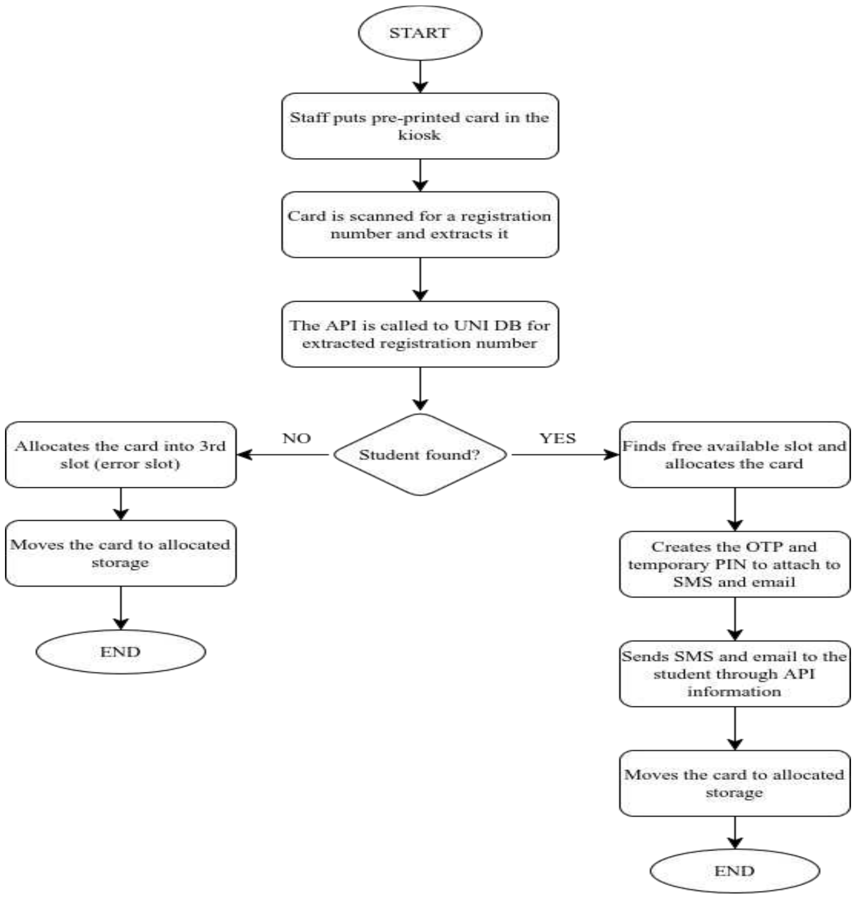


Figure 4.2: Staff-side functional workflow

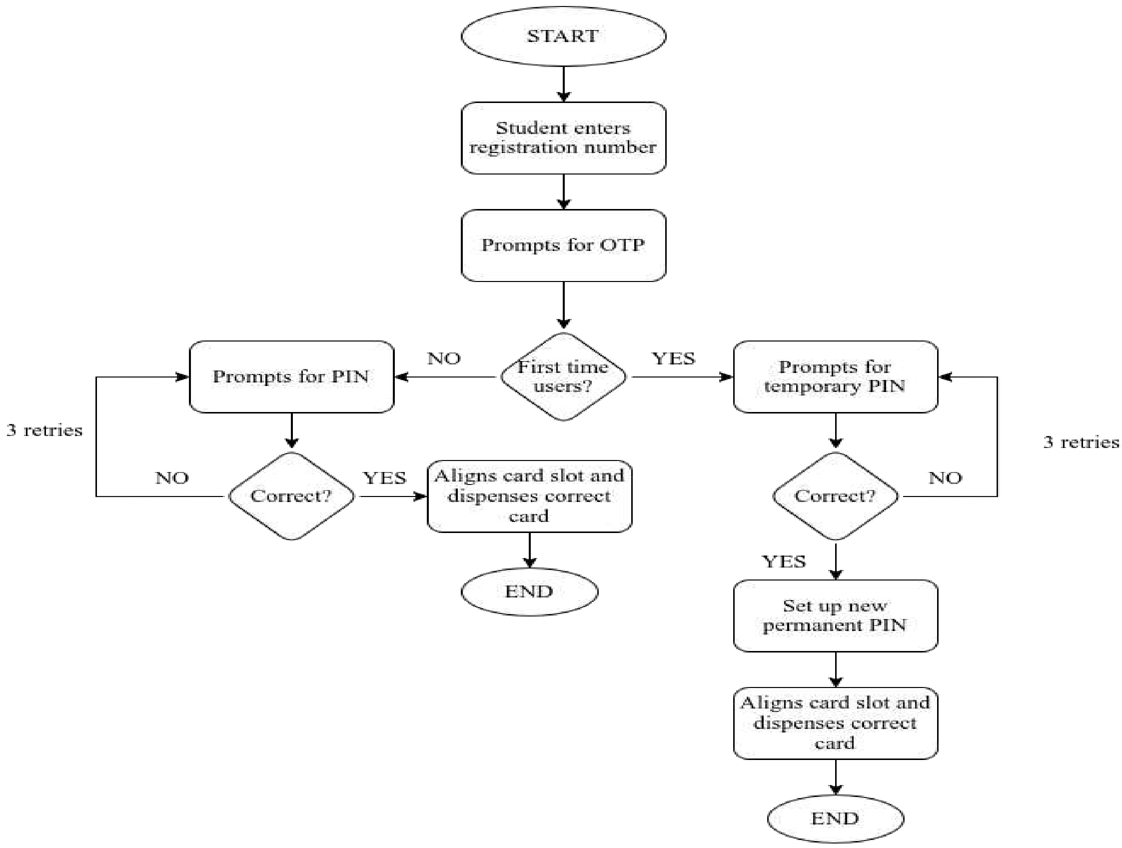


Figure 4.3: Student-side workflow

4.3 COMMUNICATION PROTOCOL

The communication protocol targeted between high-level controller and low-level controller is SPI, which has four wires (CLK, MOSI, MISO, CS) plus common ground, where Raspberry Pi is the master, with Arduino Mega as its slave. These communication signals from Raspberry Pi operate at 3.3V, while the signals from Arduino Mega operates at 5V. Since the Raspberry Pi is the master, signals at 3.3V were well – managed by the Mega counterpart. Arduino Mega does not communicate to Raspberry Pi, therefore only 3 wires used without level shifting of the voltage.

This communication protocol was selected due to high speed synchronous communication, full-duplex capability, simple hardware implementation and very reliable short distance communication.

Within this protocol, the commands (light messages) will be the ones to initiate various activities by the high-level controller through low-level controller done by the actuators.

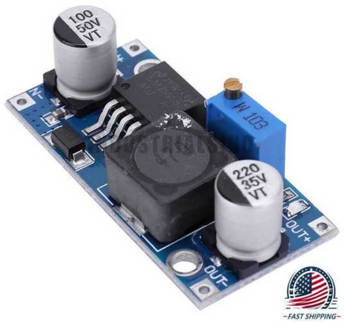
4.4 SYSTEM CIRCUIT DESIGN AND ITS ARCHITECTURE

4.4.1 COMPONENTS USED

The hardware and software tools selected for this project were chosen based on their performance requirements, need for scalability and compatibility with each other. Since the project implementation is based on the integration of software and hardware linkage, Table 4.1 below explains the function of each hardware components along with its pictorial description:

Table 4.1: Hardware components

COMPONENTS	PICTURE	FUNCTIONALITY
Raspberry Pi 5 (4GB RAM)		Hosts back-end application, OTP generation, database access, logging, computer vision and communication with low-level controller.

Arduino Mega		Directly drives actuators, reads sensors and timing and safety logic.
Buck converter		Converts 12VDC to a clean 5VDC for component usage.
Camera module V2		Visual images for card to support computer vision tasks.
Servo SG90 motor		Move sorting mechanisms, control card selection and ejection.
Power Supply		Conversion of 220VAC to suitable 12VDC for logic and control signalling.

On the software side of things, the components shown in Table 4.2 below were used to control the authentication process, database storage, and communication with hardware components. The following are the components:

Table 4.2: Software components

SOFTWARE COMPONENT	FUNCTIONALITY
Raspberry Pi OS	supporting system of the high-level controller.
Back-end Application	handles user input, authentication, interfaces with database, sends commands to low-level controller and manages error handling.
Database	stores students data and logs.
Computer Vision libraries	supports card insertion and ejection verification.
Low-level controller Firmware	executes motor control routines, reads sensors, implements communication and monitors safety.

4.4.2 HARDWARE CIRCUIT ARCHITECTURE

The hardware part of the Automated ID Card Issuance System is designed to support controlled mechanical actuation, stable power distribution and reliable communication between controllers. The circuit design was developed using Altium. The architecture design is divided into various design modules which include power regulation sub-system, Arduino Mega circuit control circuit, motor driver and actuation circuit and communication circuit interface.

4.4.3 POWER REGULATION

The system has two controllers that operate at 5V but with different current drawing capabilites. Raspberry Pi has peripherals to feed power and a hungry processor that needs up to 5A to work correctly, along with Arduino Mega that also uses the same voltage with only up to 2A with full working load. Therefore, a power circuit schematic as designed in Figure 4.4, provides the power supply for Arduino Mega and the hobby servo motors.

Due to that fact, the power concern was divided across each component individually. First, a dedicated Raspberry Pi adapter (Type – C) capable of converting 220VAC to 5VDC with 5A current rating to supply for the high – level controller. Secondly, a power supply that converts 220VAC to 12VDC was integrated with a buck converter that converts 12VDC to 5VDC with up to 3A current rating. This is designed to power the Arduino Mega and servo motors which uses 5V also.

The raw designs for buck converter using XL4015 are still presented below;

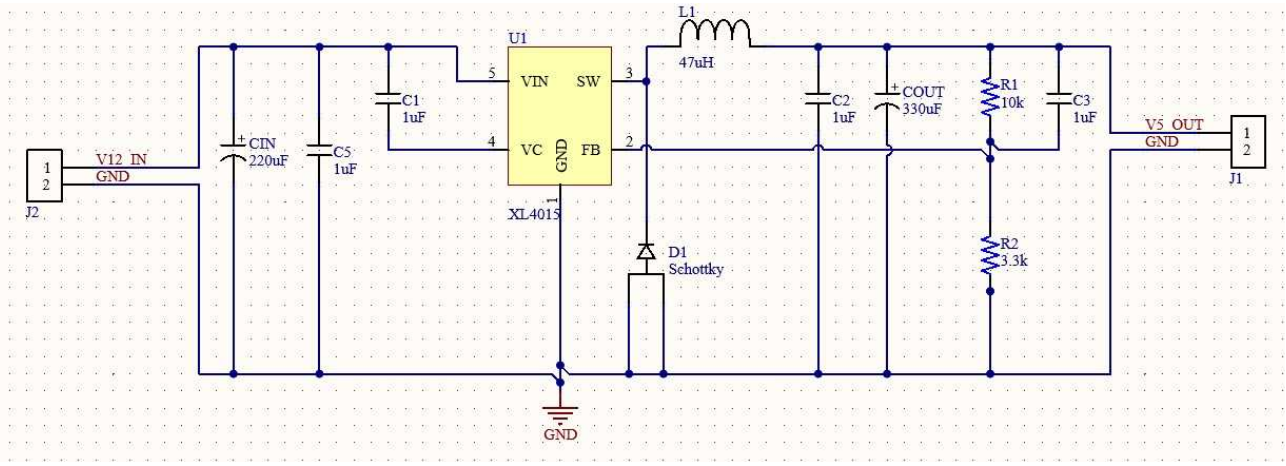


Figure 4.4: XL4015 buck converter

4.4.4 ARDUINO MEGA CORE CIRCUIT DESIGN

The Arduino Mega controller defines a real-time control of the system's actuation and deterministic behaviours. Therefore, the core design includes: proper power supply connections with decoupling capacitors to reduce power supply variations and configuration of boot pins.

Arduino Mega programming interface platform to be used will be platfromIO, together with Visual Studio Code IDE as an environment to program a low-level microcontroller used. At this stage, those applications use a way of debugging that needs a hardware (chip or board) which is not yet implemented. On the next stage of this project, programming and implementation of the project will follow along.

4.4.5 ACTUATION DESIGN

The actuation sub-system is designed to control servo motors responsible for card transport and dispensing. The motor driver circuit is designed to interface with an MCU (Arduino Mega) through various control inputs and enable logic to drive the motors efficiently.

The motor power rail is efficiently extracted from a clean conversion from a XL4015 buck converter that gives out a constant of 5V (ideally) and up to 3A that gives a larger head room since servo motors only need up to 700mA to work efficiently. All reference points from the motors, MCU, converters and sensors are to be connected to one points.

4.5 COMPUTER VISION DESIGN INTEGRATION

The computer vision module's aim is to detect and extract information from the inserted cards, either by the students or staffs. It performs OCR-based extraction of student's registration number. The extracted data was chosen to be the registration number because it is unique for each student with no duplicate complications.

4.5.1 DESIGN DETAILS

The initial barcode-verification revealed some sensitivity to orientation variation and reflection hence OCR-based design was selected to reduce dependency on alignment of the card with lights and flexibility since text are relatively larger hence easier to identify than the barcodes.

4.5.2 PROCESSING PIPELINE

The proposed system design factored the PI camera capturing images of the cards, and the high-level controller processing them to extract the registration via OCR-based detection methods. As shown in the figure 4.5 below, the captured image goes through contour based detection for card edges, perspective correction for focus, ROI fixing and finally extraction:

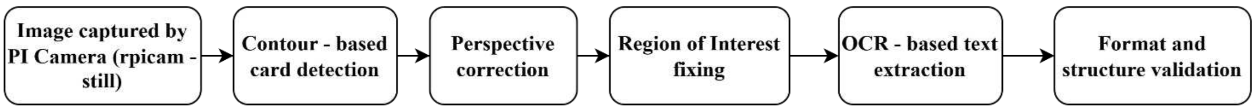


Figure 4.5: Processing pipeline workflow

From the functional workflow, card detection is designed using edge detection and contour extraction techniques which are filtered according to geometric constraints such as minimum area and aspect ratio according the CR80 standard card size. Once a contour is detected, a perspective transformation is done to obtain a clear frontal view of the card to improve OCR detection stability.

The region of interest is determined by trial-and-error fixing according to the pixel cards. The format is (x, y, w, h) coordinates.

The extracted region is processed used OCR, and configured to prioritize numeric text recognition first. Then, after extraction by OCR, the expected format of registration number "YYYY-NN-XXXXX" is validated. Only the text that fits the description of format is considered as success.

4.6 BACK-END AUTHENTICATION DESIGN

The back-end system is designed to manage identity validation, authentication control and system logging for audits. Therefore, it acts as a bridge between the computer vision results and the low-level controller activities. If the authentication goes wrong, low-level controller cannot perform any mechanical activity. The sub-system performs three important services:

- Student verification service
- OTP Authentication service
- Audit logging service.

After a successful extraction of the registration number from computer vision processes, the back-end verifies the number against the University database and fetches information about the particular student and stores the registration number to the local system's database. Only valid and active records are passed to authentication.

The main authentication mechanism is based on a One-Time Password (OTP) model. Each OTP produced is designed according to standard security principles:

- Randomized OTP generation
- Secure storage using hashing
- Time-limited validity
- Restricted number of attempts.

The OTP delivered to the student using a SMS gateway. During retrieval, the student inputs the received OTP through the user interface designed. Back-end then compares the hashed input against the stored and logs the action in either case, failure or success. The logging design ensures accountability and traceability.

4.6 MECHANICAL DESIGN

As the system requires a staff to load a pile of cards into a machine, and it sorts out each card according to the information and stores it, here is the design of it. The card sorting and storage system will be a combination of stepper motor and a circular bin calculated with each slot to an assigned mapped number stored in the system's database (Halac et al, 2025). If the card is needed, the bin is rotated by the servo motor up to the calculated position and a card is taken/ejected from the bin.

4.7 PROJECT SCHEDULE

For first semester, the following Gantt chart shows various activities planned for proposed system:

	Task	Start	End	Dur
	Design Project	24/11/25	14/3/26	80.5
1	Preliminary literature review on related projects and technologies	24/11/25	5/12/25	10
2	Field data collection on meter tampering (TANESCO)	30/11/25	13/12/25	10
3	Preparation of initial mid-semester report (old topic)	14/12/25	20/12/25	5
4	Research and evaluation of alternative project titles	21/12/25	27/12/25	5
5	Supervisor consultation on project title selection	28/12/25	1/1/26	3
6	Literature review on new project topic (automated ID dispensing systems)	28/12/25	10/1/26	10
7	Submission of new project title and follow-up consultation	4/1/26	10/1/26	5
8	Report restructuring and literature consolidation	11/1/26	16/1/26	4
9	Mid-Semester Presentation	15/1/26	16/1/26	1
10	Extended literature review (authentication, dispensing systems, embedded integration)	18/1/26	31/1/26	10
11	Comparative analysis of existing systems and technologies	1/2/26	14/2/26	10
12	Definition of system requirements and constraints	15/2/26	21/2/26	5
13	Finalizing system's block diagram and workflow	22/2/26	28/2/26	5
14	Drafting of final Semester 1 report (Introduction–Methodology)	1/3/26	14/3/26	10
15	End-of-Semester Theory Presentation	12/3/26	13/3/26	1

Figure 4.6: Semester 1 activity plan (Gantt Chart)

For second semester, the following Gantt chart shows various activities planned for proposed system:

	Task	Start	End	Dur
	Design Project	4/5/26	8/8/26	70.5
1	Procurement of hardware components	4/5/26	15/5/26	10
2	STM32 firmware development (motor & sensor control)	17/5/26	30/5/26	10
3	Raspberry Pi backend setup (OS, networking, services)	17/5/26	30/5/26	10
4	Communication setup (Raspberry Pi ↔ STM32)	24/5/26	6/6/26	10
5	Partial system integration testing	31/5/26	6/6/26	5
6	Mid-Semester Practical Presentation	4/6/26	5/6/26	1
7	Authentication implementation (OTP, registration number)	7/6/26	20/6/26	10
8	Card verification (barcode + camera integration)	7/6/26	20/6/26	10
9	Mechanical integration and actuator tuning	21/6/26	4/7/26	10
10	Fault detection, logging, and alert mechanisms	21/6/26	4/7/26	10
11	System testing under multiple scenarios	5/7/26	18/7/26	10
12	Performance evaluation and result analysis	19/7/26	21/7/26	1
13	Final report writing and corrections	19/7/26	1/8/26	10
14	Final system validation and demonstration	2/8/26	8/8/26	5
15	Final project presentation and submission	6/8/26	7/8/26	1

Figure 4.7: Semester 2 activity plan (Gantt Chart)

4.8 PROJECT BUDGET

As per logistics from local shops and imported components from AliExpress, Table 4.3 shows each component with their prices and an estimate on all the budget of the project.

Table 4.3: Project Budget

COMPONENT	ESTIMATED COST (TSh)
Raspberry Pi 5 (4GB)	290,000
Low-level controller (Arduino Mega)	60,000
Raspberry Pi Camera	67,000
Actuators (Motors and their drivers)	65,000
Sensors	25,000
Enclosure and miscallenous components	70,000
Power Supply	13,000
Interfacing modules	30,000
Perfboards and soldering equipments	55,000
Minor repairs	23,500
TOTAL	**563,500**

CHAPTER FIVE: SOFTWARE IMPLEMENTATION

This chapter depicts a complete implementation done (software) for the **Automated ID Card Issuance System.** The software parts involve the computer vision module for OCR detection and text extraction, authentication and SMS gateway configuration and user interface integration and transmission. This part of software will communicate with the hardware part which is in development for actual card sorting and ejection.

5.1 RASPBERRY PI APPLICATION

This sub-system implementation was done in a controlled Raspberry Pi OS. The focus on this was development and testing of the computer vision module and back-end authentication system together with the whole pipeline of these.

In the said raspberry Pi OS, other implementation environment and virtual environments included: python programming language, Open-CV library for image processing, Tesseract OCR engine for OCR detection, Fast API framework for back-end services and SQLite/PostgreSQL database for student records. Virtual environments were used to isolate system packages from the system's.

5.1.1 COMPUTER VISION MODULE

Implementations for computer vision module was done as follows:

- Image capturing was done by a phone camera, transferred and stored in the raspberry Pi memory for processing. This is done as a placeholder for a Pi camera V2 module which is yet to be delivered in the country.
- Card detection and perspective correction, as shown in figure 5.1 and 5.2 below, contour-based detection was implemented to identify quadrilateral shapes (card shape) and the smoothing using gaussian thresholding (adaptive) was applied to improve OCR consistency by reducing distortions. Below are the results of card detection, green lines show verification:

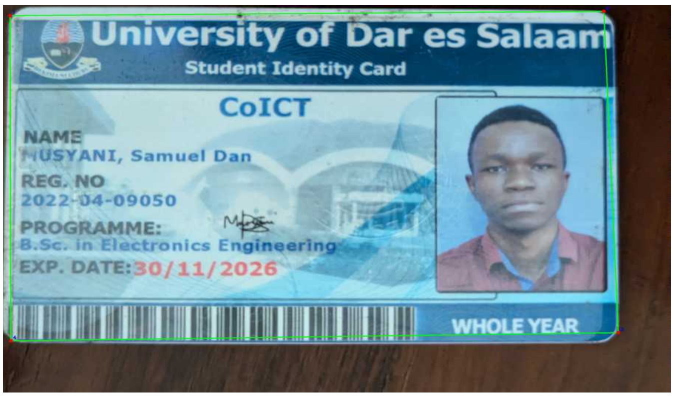


Figure 5.1: Sample card detected (green line shows detection)

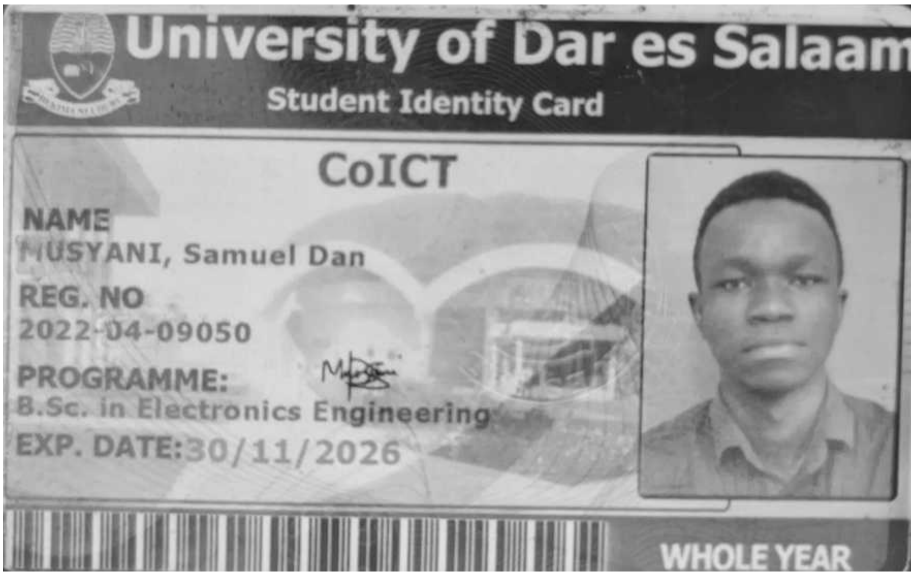


Figure 5.2: Processed grayscale sample image

- Pixel – based ROI localization, a known localized region of a standard CR-80 card was measured and calculated to attain the exact area where the registration number is located. So, with a corrected image and thresholded sample, clear characters can be extracted from a concentrated region.

  So, for a pre-ocr settings for the CR-80 card was measured through trial-and-error method and the optimum settings as shown in figure 5.3 below with height, width and the coordinate (x, y) position of the region of interest with the outputs of the settings as shown in Figure 5.4 were as:

```
ROI = {
    # default ROI for the registration number as relative fractions (x,y,w,h)
    # measured on the flattened card image (values between 0 and 1)
    "default": (0.009, 0.54, 0.28, 0.08),
}
```

Figure 5.3: ROI settings for OCR

  From the settings , there were outputs of regions of interest were observed for orignal, grayscale and threshold picture as to be shown below:

Figure 5.4: Previews of ROI outputs

- OCR extraction, validation and performance analysis, Tesseract OCR was used with prioritization of numeric recognition and validation of the extracted text against the institutional format. In addition to that, figure 5.5 shows processing time and success message that were extracted and logged into the system. The configuration settings for this are, English language, with single uniform text block (PSM 7), whitelist of alphanumeric characters only excluding the special symbols (Hyphen) in registration numbers.


```
{'confidence': 1.0,
 'error': None,
 'method': 'ocr',
 'processing_time_ms': 708.8873530001365,
 'student_id': '2022-04-09050',
 'success': True}
```


Figure 5.5: Text extraction and processing performances

5.1.2 USER INTERFACE

From the success part of the computer vision module, here comes the user interface responsible for interaction between the machine and the student in need of the card, communication with the database for authentication and validation of information and access validity together with hardware integration for physical activities such that card sorting and ejection. The UI is designed for a 5" touchscreen (TFT – Display) which will be connected directly to the raspberry Pi as a high level controller. So, the following are the screens and various transitions and conditions required:

- Idle welcome screen as in figure 5.6, this is the one that is displayed on the screen as long as the system is ON. This shows that the system is ready for student's interaction.

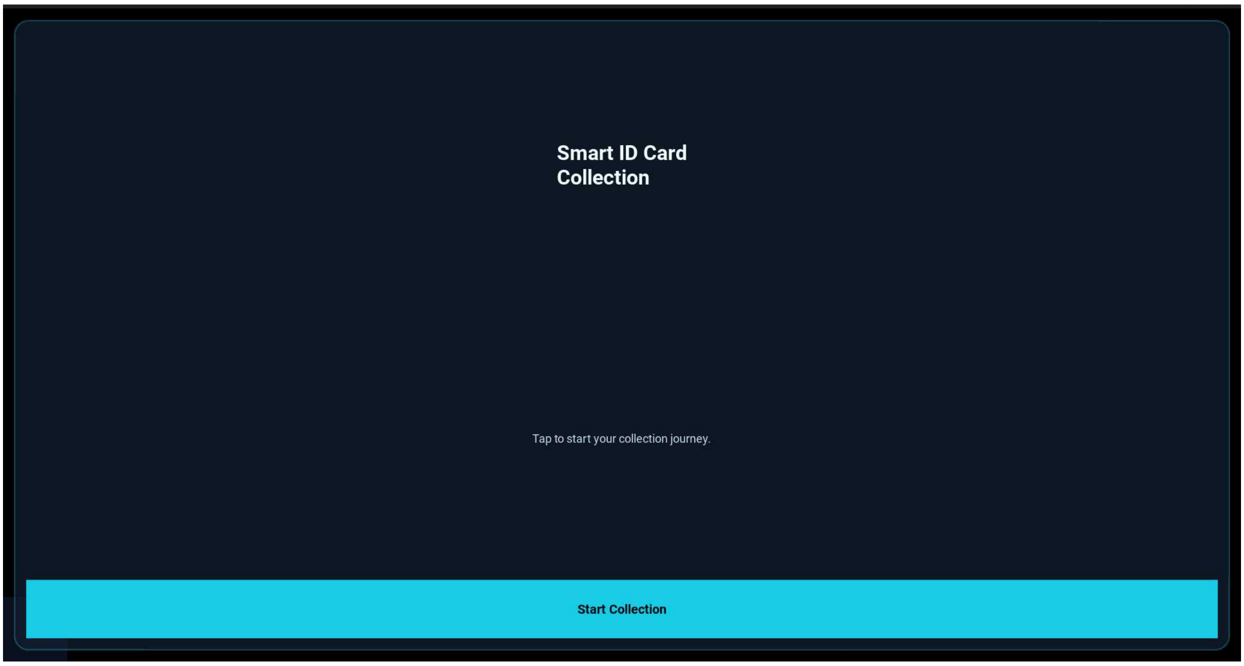


Figure 5.6: Idle UI screen for card kiosk

If a student presses the start collection button, it will transit to the next screen (registration entry screen) which prompts the student for his/her registration number.

- In figure 5.7, 5.8 and 5.9 respectively, registration number entry screen prompts the student to enter his/her registration number to which it is wired to the kiosk database. If should authenticate the student availability in the database. It checks for conditions such as if a student is not found, a card is already collected,

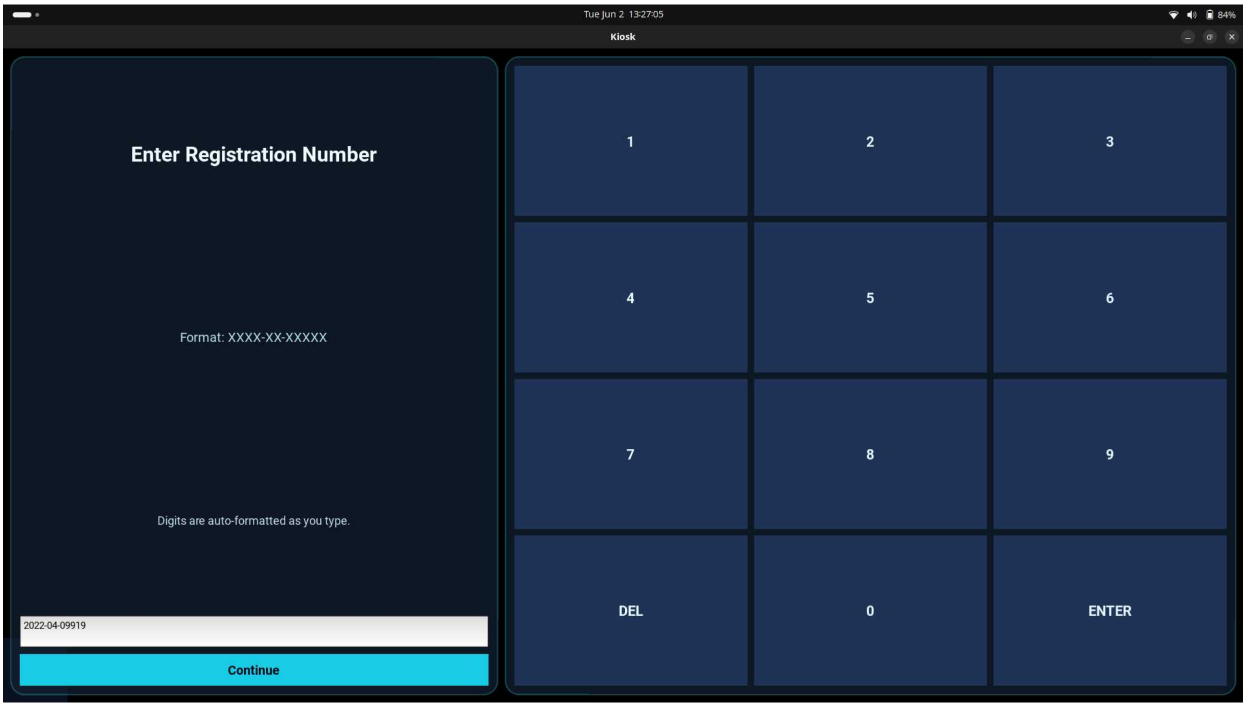


Figure 5.7: Initial registration number screen

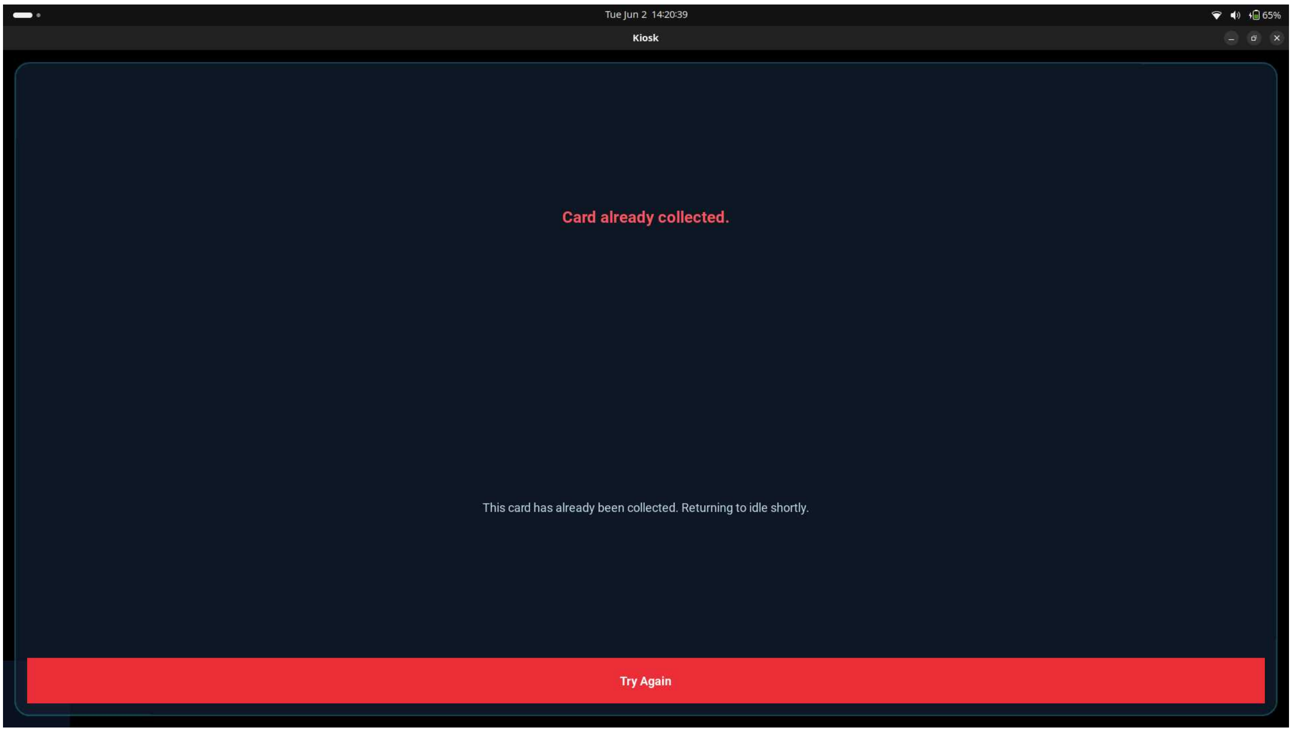


Figure 5.8: Registration entry screen for a repeated card

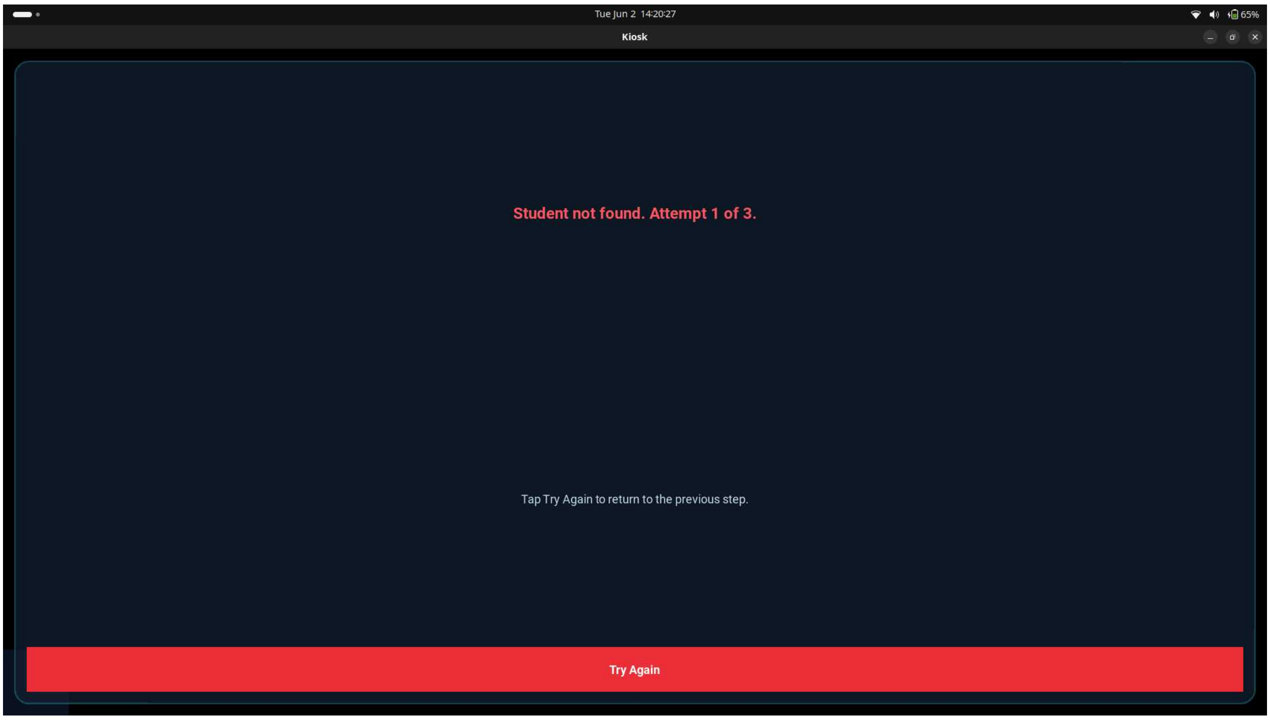


Figure 5.9: Registration entry screen for unavailable card

- OTP entry screen, student is required to enter a one time password as sent to the his/her email and phone number together as shown in figure 5.10. In figure 5.11 and 5.12 respectively, this screen validates the OTP and if correct, then the system transits to temporary PIN entry screen. If not, it warns the student and gives a maximum of three tries before the a lockout of the specific registration number.

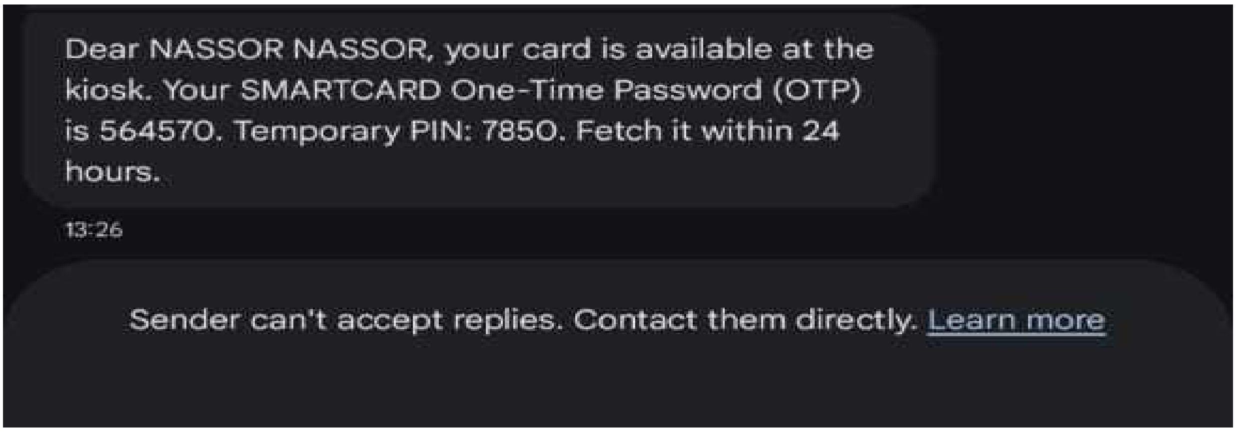


Figure 5.10: SMS message sent to sample student

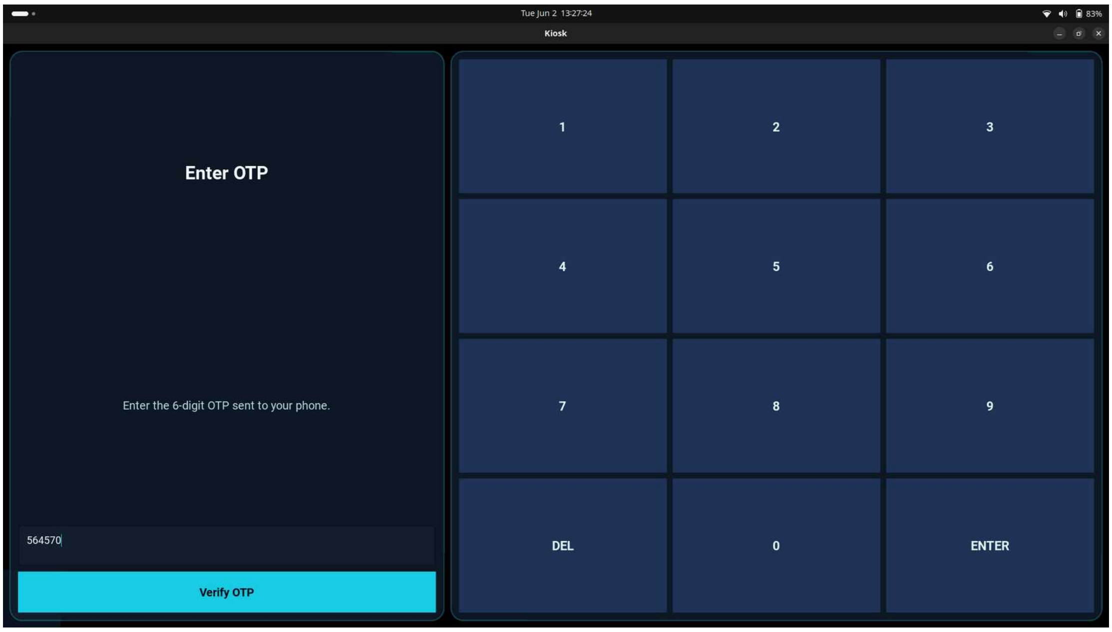


Figure 5.11: OTP initial screen

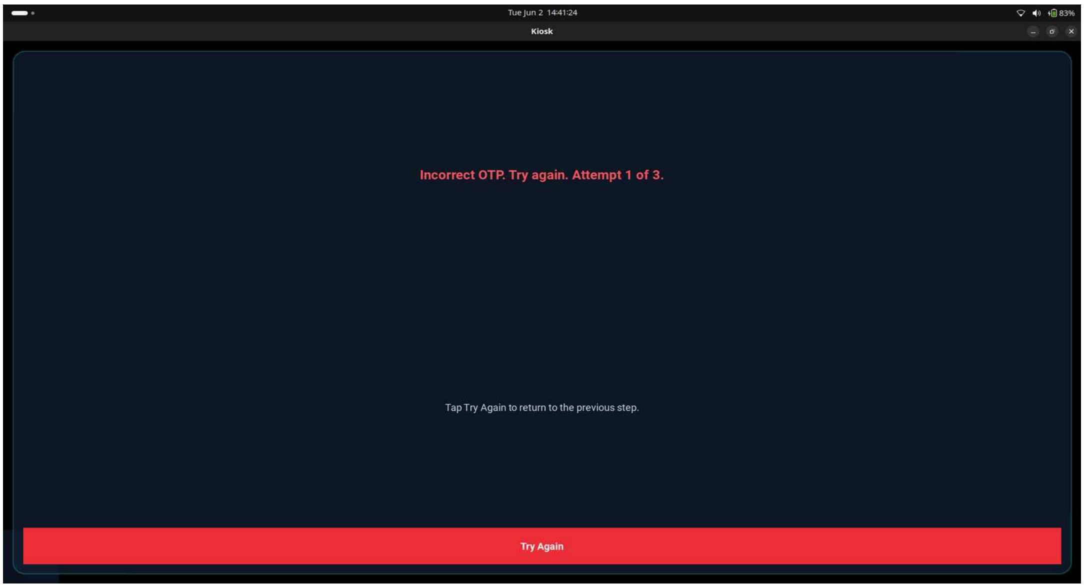


Figure 5.12: Incorrect OTP screen

- Temporary PIN screen, for first time users, the system prompts a student to enter a temporary PIN before setting their new PIN. The system uses two-factor authentication to enhance security during collection of delicate and crucial information of the student.

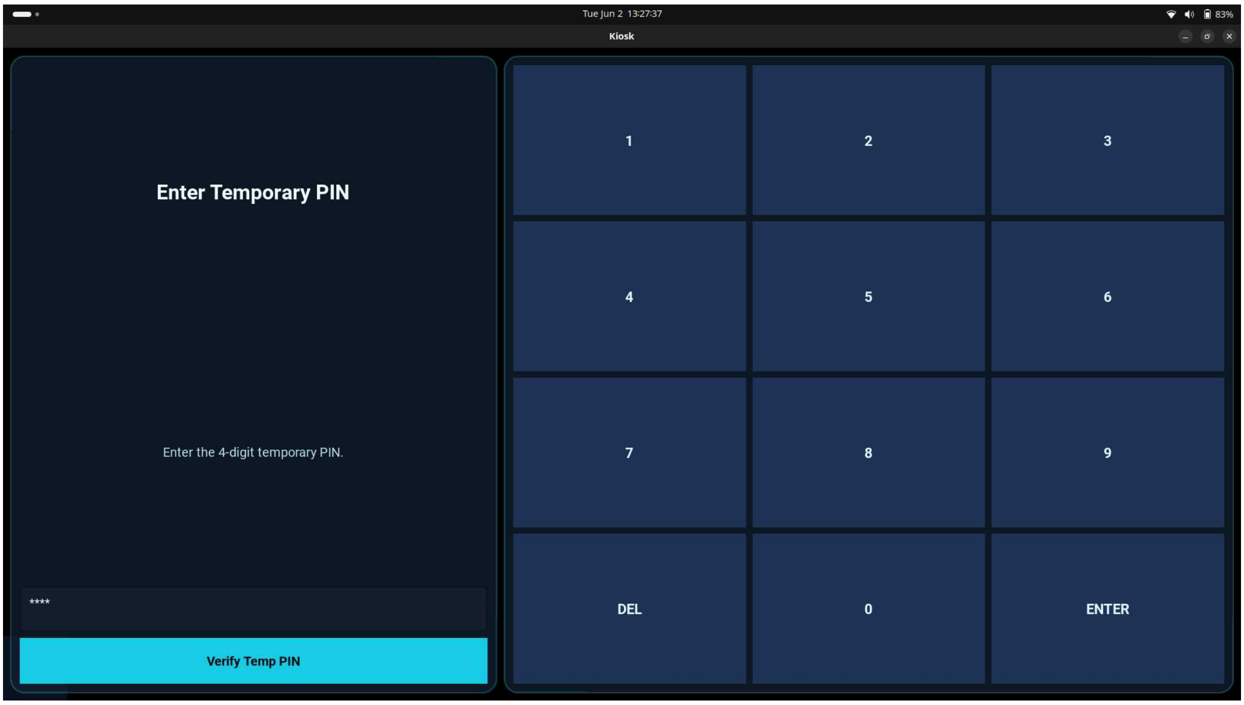


Figure 5.13:Temporary PIN screen – intial

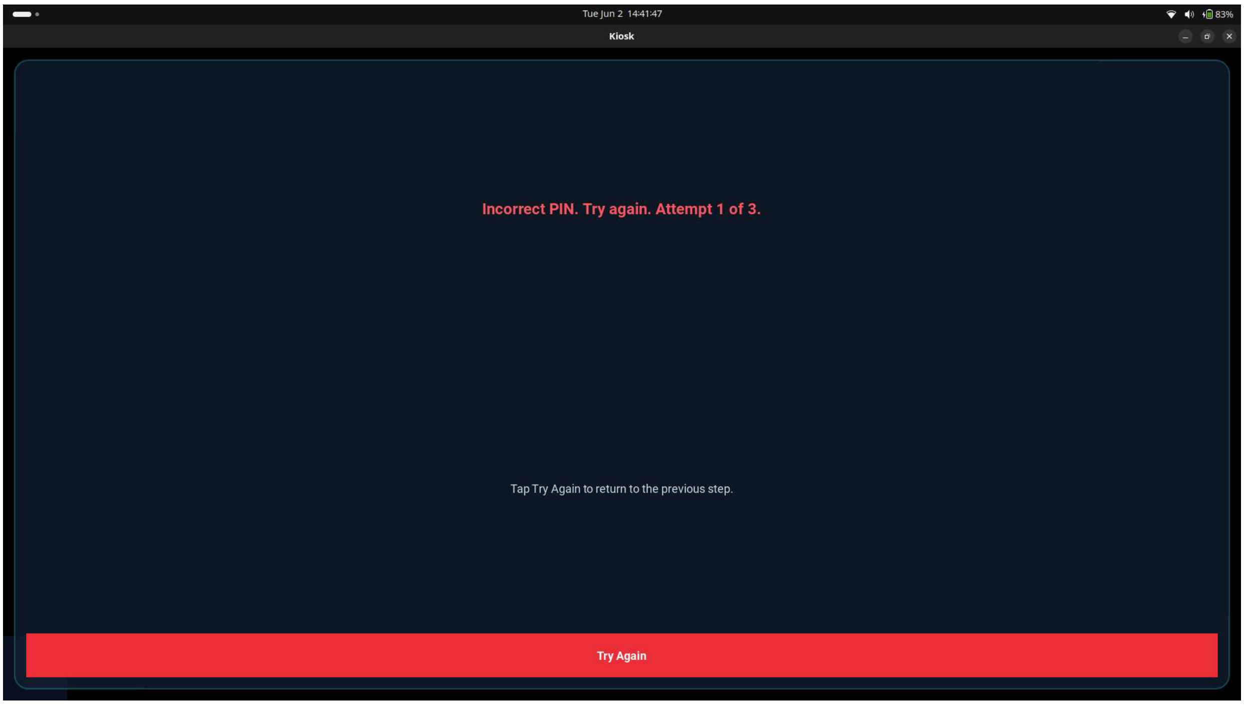


Figure 5.14: Incorrect PIN entered

- Card dispensing screen, this is a placeholder for hardware background activites. After setting the new pin, raspberry pi gives command to STM32 controller via SPI protocol to start card ejection process.

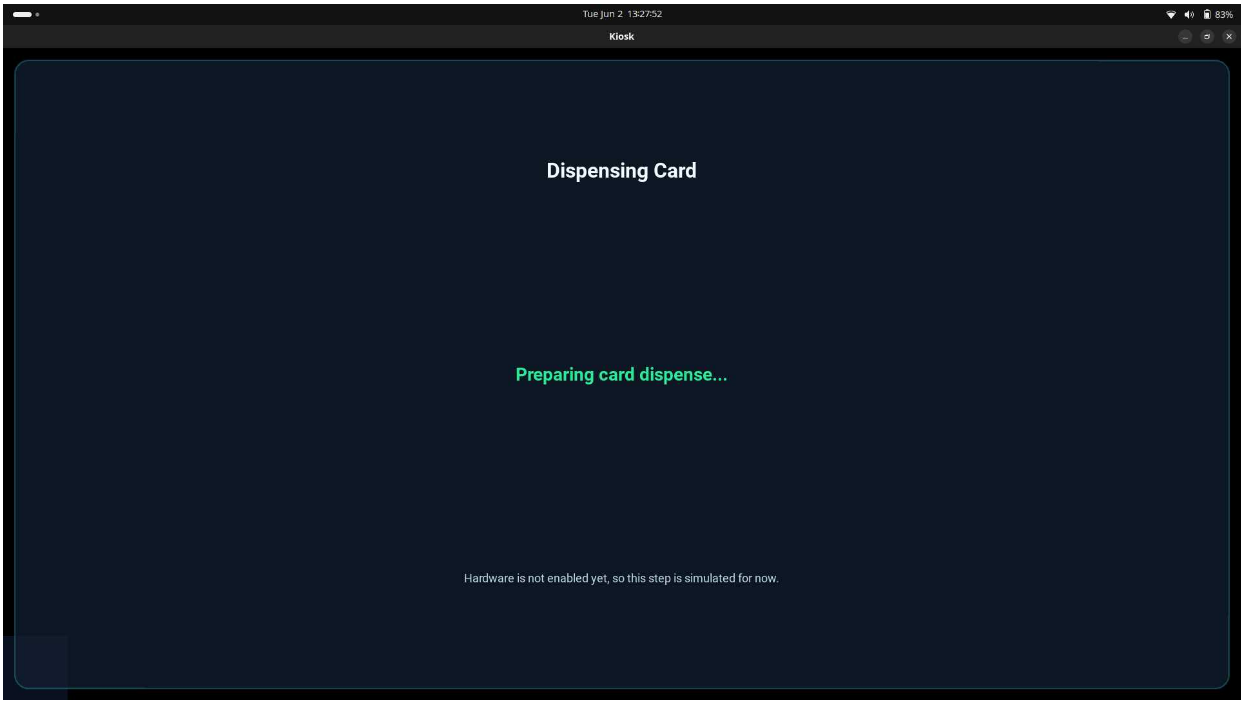


Figure 5.15: Hardware card dispensing - background

- Confirmation screen, this is the final screen that shows the student has already collected the card. This screen ends the session, and also audit all the logs into the database.

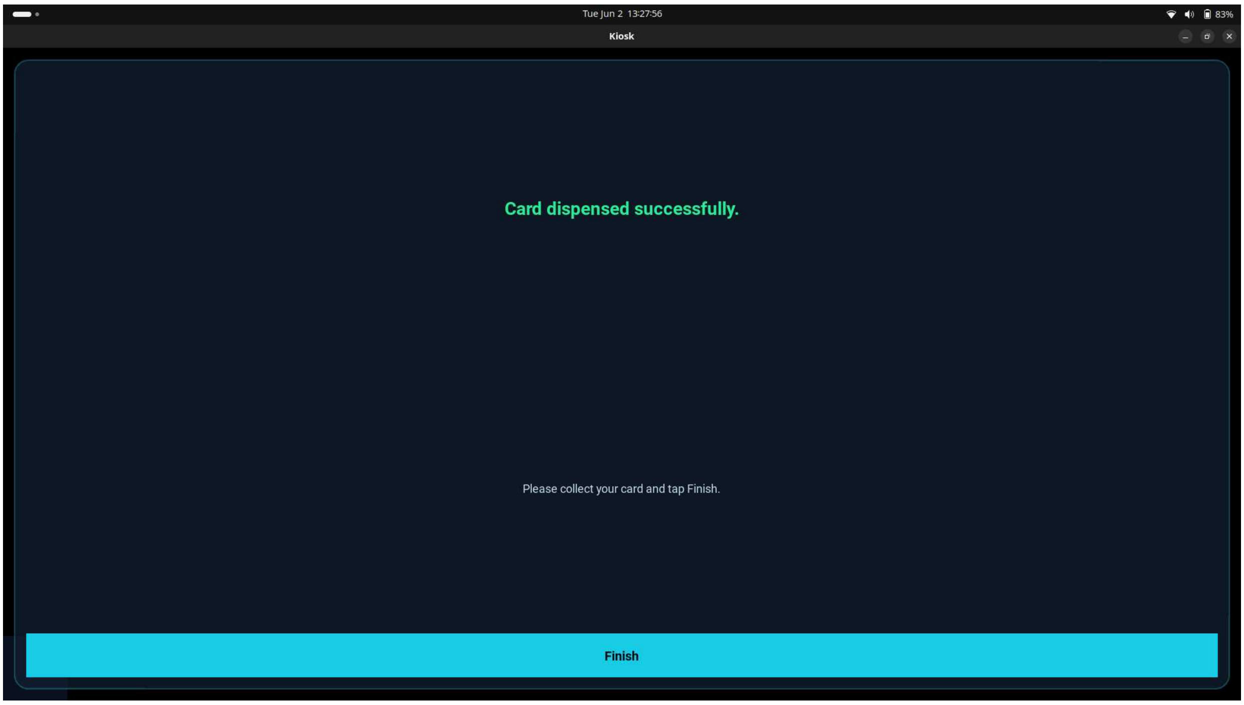


Figure 5.16: Final confirmation screen

5.1.3 OTP GENERATION AND VERIFICATION

On the implementation of OTP generation, python secrets module was used for cryptographic randomness, producing six digit numeric codes as shown in figure 5.17. In addition to that, figure 5.18 shows that the storage of these OTP codes were done by bcrypt hashing with salt rounds set to 12, preventing rainbow table attacks. The codes have 24 hour validity window from the generation timestamp, which can be changed on the main configuration file as the system admin wishes.

```
otp_number = secrets.randbelow(1_000_000)  # Random int [0, 999999]
return f"{otp_number:06d}"  # Zero-pad to 6 digits
```

Figure 5.17: OTP generation

```
return bcrypt.hashpw(
    data.encode("utf-8"), bcrypt.gensalt()
)  # Encode string to bytes, apply bcrypt hashing
```

Figure 5.18: Bcrypt hashing with salt rounds

Delivery of the OTP is distributed into email, and SMS. For SMS, BRIQ Solutions' API was used to send out messages to the required student (as the information is present in the database) while email app was used to send messages to default students' emails provided by the University.

5.1.4 UNIVERSITY API INTEGRATION AND SLOT ASSIGNMENT

The OCR extracted registration numbers are validated against the University database. The University database used was the mock database with almost the same like architecture for the real one with simplicity and not strict security just for the prototype. The endpoint called by the raspberry Pi is HTTPS GET /students/{reg_number}. Authentication used is the bearer key which is also found in the configuration files. Timeout for the API call is 5 seconds with exponential back off retries with a maximum of 3 attempts.

The said API call handles file names such as first_name, surname, phone_number, email and other like personal information.

Upon successful validation, the system assigns an active slot on the round storage platform. The Round-robin allocation among three active slots (0-2) which are scalable. The application looks for a free slot in the database (slot_index) and puts the scanned card into the slot. The card scanned is linked to scan timestamp data and batch ID which are also stored in the database.

5.2 DATABASE ARCHITECTURE – KIOSK LOCAL

The SQLite database has data with five normalized tables designed for audit compliance and query efficiency.

Table 5.1: Schema overview

TABLES	COLUMNS	PURPOSE
Students	id (PK), registration_number (UNIQUE), first_name, last_name, email, phone, programme, status	Student master records; single source of truth for student information
Authentication	d (PK), student_id (FK), otp_hash, otp_generated_at, otp_verified_at, temp_pin_hash, permanent_pin_hash, is_temp_pin, failed_otp_count, failed_pin_count, otp_lockout_until, pin_lockout_until, last_activity	Credential storage with bcrypt hashes; lockout state management
Cards	id (PK), card_number (UNIQUE), student_id (FK), slot_index, batch_id (FK), status (pending_collection or collected or rejected), created_at, collected_at	Physical card tracking; links students to assigned dispenser slots
Audit Log	id (PK), registration_number, session_id, event_type (registration or otp_generated or otp_verified or pin_verified or card_collected or error), failure_reason, timestamp	Immutable append-only log for regulatory audit trail
Batches	id (PK), batch_name, staff_id, card_count, scanned_at	Metadata for batch ingestion events

For this presented schema overview, the indexing strategy in those columns goes as follows; in students table, registration numbers for fast student lookup during collection, in audit log, chronological log traversal and for cards, student_id and status for query pending cards for given students.

5.3 API CLIENT MODULE AND SMS DELIVERY

From the application, a module dedicated for api client activites abstracts communication with the mock university database. The key features are as follows:

- mDNS discovery: automatically resolves university-db.local (my computer for this prototype) on startup with a requirement of Avahi daemon package
- Bearer key, a security configurable API key in the configuration file
- Timeout handling, 5 second timeout with exponential backoff with maximum of 3 attempts.
- Fallback caching, this returns the cached student records if the network is unavailable.

After the OTP generation, the system was configured to use BRIQ Solutions, a Tanzania – based SMS gateway with the following configuration requirements:

- BRIQ API KEY: authentication token from one's own BRIQ account.
- BRIQ SENDER ID: sender name to be displayed to the reciever. For this project, the sender ID is ***CARD KIOSK***
- BRIQ BASE URL: endpoint that communicates the application and the BRIQ's dedicated account.

From all these configuration, sending message from the application to student's personal number retrieved from the Universtiy database is made possible. The following are the steps towards sending the SMS:

i. Construction of a message with OTP generated with a temporary PIN for first users.

ii. POST request to BRIQ "/v1/message/send-instant" with a referenced API key.

iii. Log the delivery status to the audit log table

iv. On failure, queue for retry every 15 minutes (up to 3 attempts)

Credential delivery is only considered successful when at least one channel (SMS or email) proceeds. This means that the receiver has got the OTP successfully.

5.4 SESSION MANAGEMENT

The session manager class maintains per transaction states in volatile memory (RAM). Initialization of a session is done on the registration number entry on the UI, with state varibales such as validated flags on OTP and PIN, assigned slot index, and student ID. Teardown is done on the end of transaction or inactivity for a timeout of 60 seconds. The session ID is logged into the audit log for transaction correlation.

5.5 CONFIGURATION MANAGEMENT

In the raspberry Pi application, a dedicated module that centralized all configurable parameters is implemented but isolated from the version control due to presence of critical secrets such as API KEYS, and passwords. The parameters contained in there are:

- Database: DB_PATH, connection for the database
- APIs: UNIVERSITY_API_BASE_URL, UNIVERSITY_API_KEY, timeouts, retry policies
- SMS: BRIQ_API_KEY, BRIQ_SENDER_ID, BRIQ_BASE_URL
- Email: SMTP_EMAIL, SMTP_PASSWORD (app-specific, not account password)
- Camera: Device candidates, capture resolution (1920×1440), frame rate, thresholds
- OCR: Perspective correction size, registration ROI coordinates, preprocessing values
- UI: Timeout intervals, attempt limits, lockout durations

CHAPTER SIX: HARDWARE IMPLEMENTATION

6.1 MICROCONTROLLER SELECTION AND CONFIGURATION

The current project hardware implementation targets Arduino Mega 2560, a 16MHz AVR based micro controller as shown in the figure 6.1, with the following hardware specifications:

- Clock speed: 16 MHz crystal oscillator
- Memory: 256 KB Flash, 8 KB SRAM, 4 KB EEPROM
- GPIO Pins: 54 digital I/O, 16 analog inputs
- Hardware Peripherals: SPI (slave mode), UART (serial monitor), PWM timers
- Programming: PlatformIO framework with AVR HAL abstractions in Visual Studio Code IDE

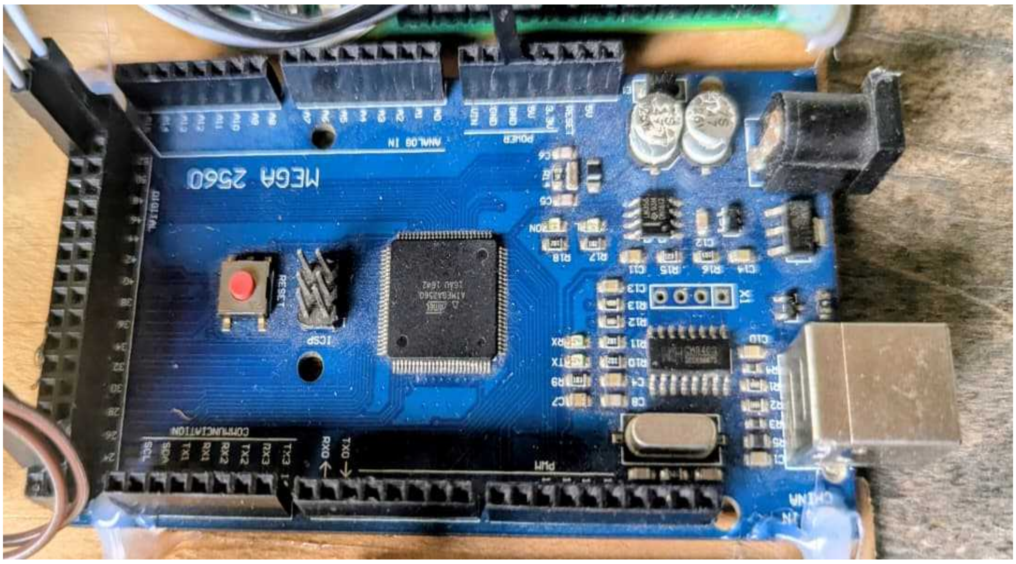


Figure 6.1: Arduino Mega micro-controller

The original microcontroller choice was STM32 Nucleo-F401RE with 1MHz SPI, but this substitution due to logistics maintains compatibility through platformIO abstraction with an improvise of speed for SPI at 100 kHz.

6.2 SPI SLAVE CONFIGURATION

The micro-controller operates as a slave (SPI), receiving two-byte ASCII frame commands from the raspberry Pi master. The pin mapping of the connection configuration is as follows:

- SS (Slave Select): pin 53 (active low), with pin 24 from Raspberry Pi
- MOSI (Master Out Slave In): pin 51, with pin 19 from Raspberry Pi
- SCK (Serial Clock): pin 52, with pin 23 from Raspberry Pi

With considerations of different operating voltages between Raspberry Pi and the arduino Mega, since the slave does not send any signal to its master, hence no level shifting is required.

6.3 SPI COMMUNICATION PROTOCOL

The current implementation uses a two-byte ASCII frame format:

- Byte 0: command character (0 – 9, A – Z)
- Byte 1: parameter character (0 – 9, slot index)

Example status frames are:

- “00”: processing error
- “1X”: 1 – success, X – slot index
- “2X”: 2 – retrieve, X – slot index

The current firmware implements basic frame reception and servo test routines.

6.4 SERVO MOTOR CONTROL

The firmware configures hardware PWM outputs for servo motor controls. The servo type in conversation is SG90 with 180 degrees physical constraints.

- Pin 22: card holder for scanning card during entrance
- Pin 23: carousel rotation motor controlled

Figure 6.2: Servo motor attached to card holder

For servo control, pulses vary from 544 μs to 2400 μs, to rotate from 0 to 180 degrees.

6.5 FIRMWARE ARCHITECTURE

The arduino Mega firmware follows a cooperative multitasking pattern with SPI hardware interrupts. The main loop structures goes as follows:

- Poll SPI SPIE flag for received framework

```
ISR(SPI_STC_vect) {
  char received = static_cast<char>(SPDR);

  if (spiFrameIndex < SPI_FRAME_LEN) {
    spiFrame[spiFrameIndex++] = received;
    if (spiFrameIndex == SPI_FRAME_LEN) {
      spiFrame[SPI_FRAME_LEN] = '\0';
      spiFrameReady = true;
      spiFrameIndex = 0;
    }
  } else {
    spiFrameIndex = 0;
  }
}
```

Figure 6.3: SPI poll initialization

- Extract two-byte command and parameter from SPI buffer

```
bool readSpiFrame(char *out, size_t outSize) {
  if (out == nullptr || outSize < (SPI_FRAME_LEN + 1)) {
    return false;
  }

  noInterrupts();
  bool ready = spiFrameReady;
  if (ready) {
    out[0] = spiFrame[0];
    out[1] = spiFrame[1];
    out[2] = '\0';
    spiFrameReady = false;
  }
  interrupts();

  return ready;
}
```

Figure 6.4: Frame extraction from buffer

- Dispatch command to state machine

```
bool receiveSpiMessage(char *out, size_t outSize) {
  if (!readSpiFrame(out, outSize)) {
    return false;
  }

  Serial.print("Received SPI: ");
  Serial.println(out);
  return true;
}
```

Figure 6.5: Dispatch to state machine

- Drive servo motors according to the received SPI buffer.

```
if (receiveSpiMessage(out: spiMessage, outSize: sizeof(spiMessage))) {

  if (strcmp(s1: spiMessage, s2: "10") == 0) {
    Serial.println("Target: Compartment A -> Entrance (0 deg)");
    servoToAngle(&servo: servo2, angle: 0);
  }
  else if (strcmp(s1: spiMessage, s2: "00") == 0) {
    Serial.println("Target: Compartment C -> Entrance (90 deg)");
    servoToAngle(&servo: servo2, angle: 90);
  }
  else if (strcmp(s1: spiMessage, s2: "11") == 0) {
    Serial.println("Target: Compartment B -> Entrance (180 deg)");
    servoToAngle(&servo: servo2, angle: 180);
  }
  else {
    Serial.print("Unknown Frame: ");
    Serial.println(spiMessage);
  }
}
```

Figure 6.5: Choice statements

6.6 SYSTEM INTEGRATION

On the hardware part, SPI communication protocol made integration between the raspberry Pi and arduino Mega communicate well. As shown in figure 6.7 below, all connections were referenced to the ground for cleaner signal transmission. Also, a whole round project enclosure with a circular carousel storage, a stand for holding the phone to be used for capturing images, a card holder driven by a servo motor together with the control system that involves a high level controller, low level controller together with a buck converter which produces an output of 5VDC with a 12VDC input.

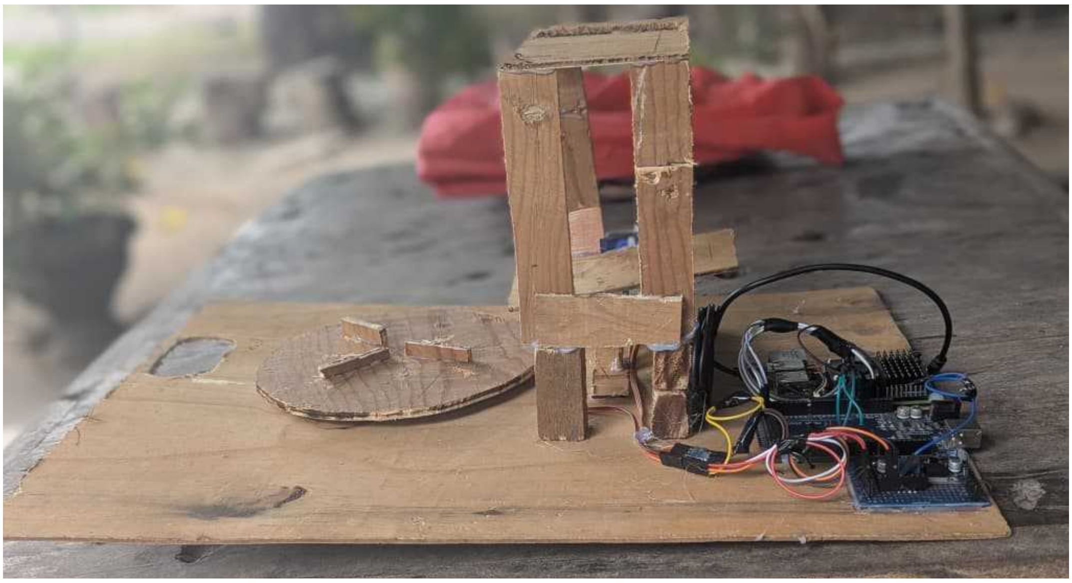


Figure 6.6: System integration

CHAPTER SEVEN: RECOMMENDATIONS AND CONCLUSION

7.1 CONCLUSION

The Smart ID Card Issuance Kiosk project demonstrates a practical application of embedded systems, real-time control, and distributed computing to address operational challenges in higher education. The implementation achieves functional completeness at the application and database layers, with a foundational SPI protocol and microcontroller framework ready for production hardware integration.

Table 7.1: Performance metrics of the system

PERFORMANCE METRIC	UNITS	EXPLANATIONS
Average card retieval time	20 – 30 seconds per card	The estimated time covers registration-number entry, database verification, OTP and PIN authentication, card-slot identification, controller communication and physical card ejection. Approximately 6–9 seconds may be used for student authentication, 1–3 seconds for database processing and 10–17 seconds for card positioning and dispensing.
Authentication success rate	> 85%	A valid student should be successfully verified using the registration number, OTP and PIN. The remaining percentage allows for failures caused by incorrect credentials, expired OTPs, message-delivery delays or temporary network problems
Authentication processing time	6 – 9 seconds per attempt	This is the estimated system-processing and user-entry time required to check the registration number, OTP and PIN. The exact time depends mainly on how quickly the student enters the required information.
Error – handling success rate	> 90%	The system is designed to identify errors such as an unknown student, unavailable card, already collected card, incorrect OTP, incorrect PIN, exceeded attempts and communication failure.

		Successful handling means displaying the correct message, stopping an invalid transaction and recording the event
Error – response time	1 – 3 seconds per error	Errors related to registration numbers, OTPs, PINs and database records are expected to be detected shortly after the student submits the information. Mechanical and communication errors may require slightly more time
Card dispense rate	> 90%	A successful attempt means that the correct storage slot is selected, only one card is released, the correct card reaches the collection point and the database is updated correctly. The estimated value allows for possible card jams, alignment problems or incomplete motor movement during prototype testing.
Physical card – dispensing time	8 – 10 seconds per card	This is the estimated time required for the Arduino Mega to receive the command, move the storage chamber to the assigned slot and activate the card-ejection mechanism.
Overall system conclusion	Functional prototype with estimated performance targets	The prototype has implemented student verification, OTP and PIN authentication, database management, error handling, card-slot identification, audit logging and communication between the Raspberry Pi and Arduino Mega.

7.2 RECOMMENDATIONS

Considering the status and the risk of Implementation, I suggest the first prototype be installed at the main university campus rather than separate college locations. This recommendation is driven by three technical and operational considerations. First, the system has achieved 95% software completeness but about 65% mechanical hardware integration, requiring proximity to development resources for troubleshooting the carousel homing sequences, SPI timing validation, and calibration of sensor feedback. Second, the main campus has the largest student population, providing a sufficiently large transaction volume for validation of authentication workflows, detection of OCR failure modes under realistic variations of the lighting and card conditions, and stress testing of the SQLite database under peak concurrency conditions. Third, the centralized deployment allows iterative refinement of the core system prior to deploying replicated instances across geographically dispersed college campuses.

But deployment on the main campus should not mean permanent centralization. After 90 days of operation without any critical hardware failures I would suggest rolling out a second kiosk to the largest isolated college campus. This secondary deployment is a validation point for the consistency of the multi-instance architecture: database replication needs, API caching strategies, network failure scenarios, and configuration management patterns are not derivable from a single kiosk in ideal circumstances. The geographically separate college location naturally isolates future enhancement testing, such as the printer workflow discussed below, from main campus operations and user experience.

In addition to reducing mechanical complexity during the crucial hardware integration window, the present prototype dispenses cards without access control barriers, which is suitable for validation. Similarly, once motor control, sensor feedback, and slot homing are demonstrated to be dependable in production, expanding storage capacity (e.g., from four active slots to ten or adding a second carousel tier) is a straightforward mechanical modification. On-demand card printing is the higher-value Phase 2 project. Once the SPI dispensing protocol is established and the current system consistently connects students with physical pre-printed cards, retrofitting a thermal or inkjet printer into the dispensing assembly eliminates the need for an OCR pipeline, completely eliminates staff batch-loading workflows, and drastically lowers administrative overhead.

REFERENCES

- Aman Prasad, Shraddha Kumar, Keshav Shriram, Prof. Dr. Vikram Mane, "IOT Based Pen Vending Machine", Published in International Research Journal of Innovations in Engineering and Technology-IRJIET, Volume 8, Issue 3, pp 367-373, March 2024. Article DOI https://doi.org/10.47001/IRJIET/2024.803057
- Bai, Y., Hu, J., Liu, J., & Zhou, Q. (2012). *Card transport device and ATM using same* (U.S. Patent No. US8251282B2). U.S. Patent and Trademark Office.
- Halac et al. Journal of Engineering and Applied Science, (2025) 72:187. https://doi.org/10.1186/s44147-025-00763-0
- https://docs.briq.tz/karibu
- Instructables User "Joe_Squared". (2017). *DIY trading card dispenser project*. Instructables.
- Mithun Jaya Kumar, Nanditha V, Pranav P Nair, Sona S, Ann Mary Alex, 2025, Library Management Robot with Line Following Navigation and RFID Based Book Identification, INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT) Volume 14, Issue 03 (March 2025),
- Peter Kibaya, Charles S Lubobya (2024) Design of a Multifactor Authentication System for Automated Teller Machines. 1: 1-17
- Ratnasri, Nilani & Sharmilan, Tharaga. (2021). Vending Machine Technologies: A Review Article. International Journal of Sciences: Basic and Applied Research (IJSBAR). 58. 160-166.
- Xia, K., Fan, H., Huang, J., Wang, H., Ren, J., Jian, Q., & Wei, D. An Intelligent Self-Service Vending System for Smart Retail. *Sensors*, *21*(10), 3560. https://doi.org/10.3390/s21103560